Machine Learning for Experimental Design Optimisation

Conditional Neural Bayes Ratio Estimation (cNBRE)

Sam Leeney

with Thomas Gessey-Jones, Will Handley, Eloy de Lera Acedo, Harry Bevins, Jacob Tutt

AI in Radio Astronomy Workshop, Royal Observatory Edinburgh

Thursday 19 March 2026

Why is experimental design still decided like this?

"How long should this cable be?"

Many design decisions in experimental physics are set by habit, catalogue availability, or a very small parameter sweep.

In sensitivity-limited experiments, these choices can determine whether the signal is detected at all.

This talk: use machine learning to simulate, forecast, and optimise *before* building.

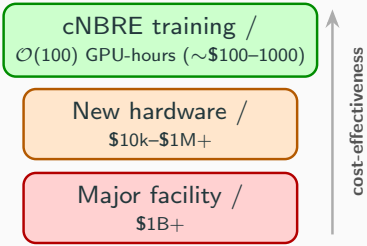
What actually needs optimising?

These are all **design parameters** – they impact the effectiveness of our system:

- Observation time: τ
- Antenna orientation: ψ
- Antenna geometry and placement
- Cable lengths and calibration hardware
- Component tolerances

Main teaser

Antenna orientation alone changes forecast detection probability by about **20 percentage points** — and costs **nothing** to implement.



Increase sensitivity for a fraction of the cost of new hardware.

Outline

Method

Results

Applications

Method

What quantity do we need for detection?

We quantify confidence in a detection by comparing the Bayesian evidence for a signal model versus a null model:

$$M_1 : \text{signal} + \text{foreground} + \text{noise}$$
$$M_0 : \text{foreground} + \text{noise}$$

$$K(\mathbf{d}) = \frac{p(\mathbf{d} \mid M_1)}{p(\mathbf{d} \mid M_0)}.$$

- $K > 1$: data prefer the signal model.
- $K \gg 1$: strong evidence for the signal.
- If I can compute K, I can tell which design gives the best chances of detection.

The bottleneck

Optimising a design parameter requires many samples over the full design space.

Each sample is an independent sampling run.

100 design choices $\times$ 100 possible signals = **10,000** expensive runs.

Add one more parameter and it becomes **1,000,000**.

How do evidence networks avoid that bottleneck?

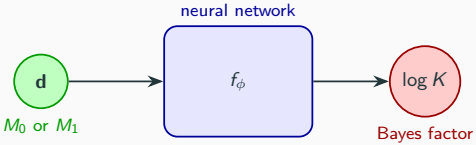
If we have a simulator for M_1 and M_0, we can train a classifier $f_\phi(\mathbf{d})$ to distinguish them.

With an appropriately chosen loss function, the network converges on the Bayes factor:

$$\mathcal{L} = \mathbb{E}_{M_1}\left[e^{-f_\phi(\mathbf{d})/2}\right] + \mathbb{E}_{M_0}\left[e^{+f_\phi(\mathbf{d})/2}\right].$$

$$f_\phi^\star(\mathbf{d}) = \log K(\mathbf{d}).$$

- Feed the network different data, it outputs the Bayes factor.
- Train once, evaluate in milliseconds.



Optimised f_ϕ^* predicts $\log K$ from data $\mathbf{d}$.

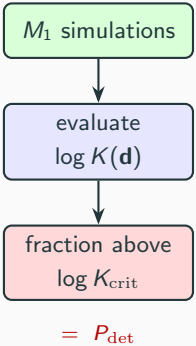
How does this become forecasting?

Given some range of physics parameters, what is our probability of detecting the signal?

Gessey-Jones et al.'s key insight was to use the Bayes factor as a forecasting utility:

$$P_{\mathrm{det}} = \Pr\left[\log K(\mathbf{d}) > \log K_{\mathrm{crit}} \,\middle|\, M_1\right].$$

- Simulate many realisations under M_1.
- Evaluate $\log K$ for each one.
- Count the fraction that cross the threshold.

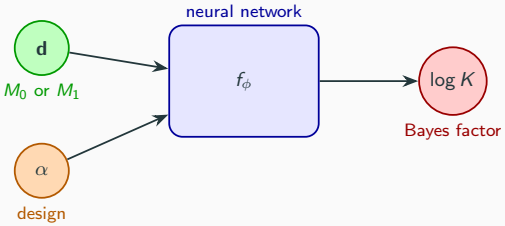


What changes in conditional neural Bayes ratio estimation?

Extend the classifier: predict which model the data came from **given some design parameter**:

$$f_\phi(\mathbf{d}, \alpha) \approx \log K(\mathbf{d} \mid \alpha).$$

- e.g. "did this data come from the signal model, **given some cable length**?"
- Train once across the full design space.
- Query **any** α in milliseconds.



Key idea

Amortised: check how ANY design choice impacts detection odds.

Application to 21-cm cosmology with REACH

A useful testing ground: tough, unsolved problem with dim signal compared to foreground and very advanced, fast simulators.

- Signal amplitude: about **100 mK**
- Foregrounds: 10^4–10^5 **times brighter**
- Forward model: physics-based and implemented in JAX

If cNBRE works here, it is worth taking seriously elsewhere.

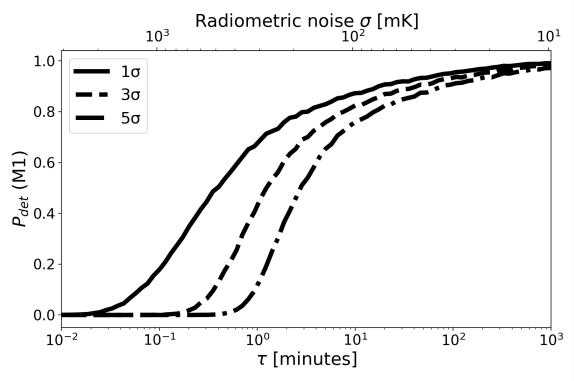
$$M_1 : \mathbf{d} = \mathbf{d}_{\mathrm{fg}} + \mathbf{d}_{21} + \mathbf{n}$$

$$M_0 : \mathbf{d} = \mathbf{d}_{\mathrm{fg}} + \mathbf{n}$$

REACH

Results

Does cNBRE recover the right physics?



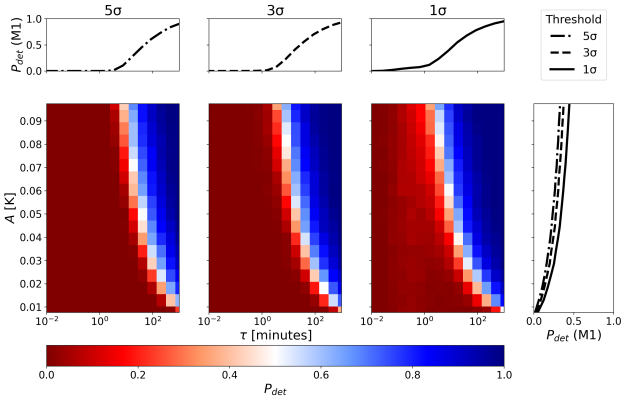
We start on a simple problem where we know the answer: observation time as the design parameter.

- As τ increases, noise goes down, so detection should get easier.
- The trained network tells us: "your probability of detecting the signal at this observation time is X".
- The network learns the radiometer-equation scaling.

Validation

The network reproduces the basic physical behaviour we expect. It does.

Amortised: amplitude × observation time

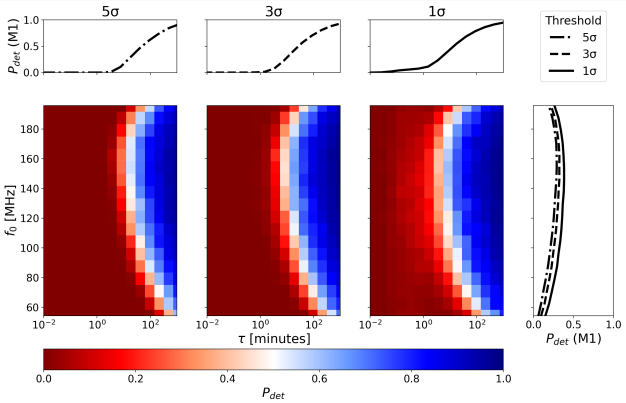


Amortised exploration

Given some range of **physical signal parameters**, what is our P_{det} given our **design parameter**? **Same network, no retraining.**

- Each pixel would have required a separate nested sampling run.
- The joint map shows how instrument choices and the underlying physics interact.
- Ask about amplitudes today, signal *locations* tomorrow – just re-slice.

Amortised: frequency × observation time



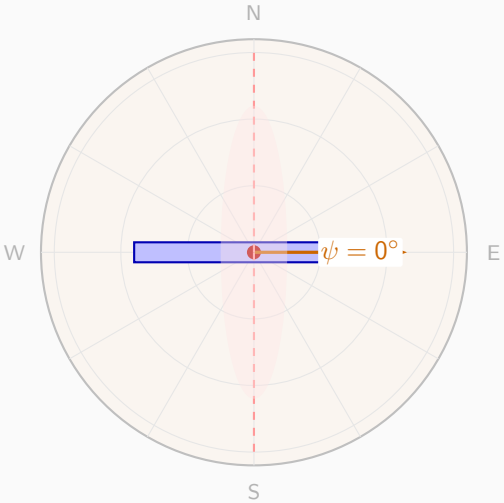
Same network, different slice

Answer a new question without re-training, in less than 1 second.

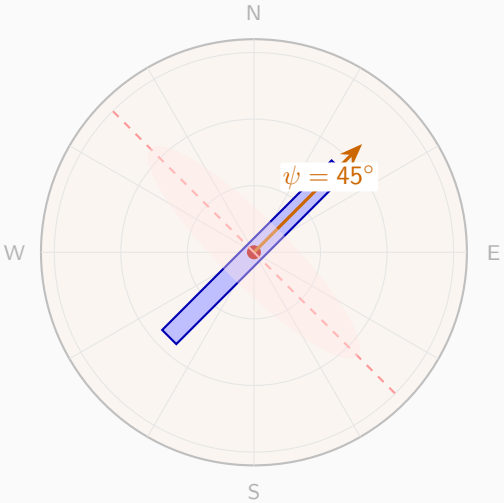
Signals at 100–140 MHz are easier to detect; sensitivity drops at band edges.

- This is the **same trained network** as the previous slide.
- We just sliced down a different axis of the detection landscape.

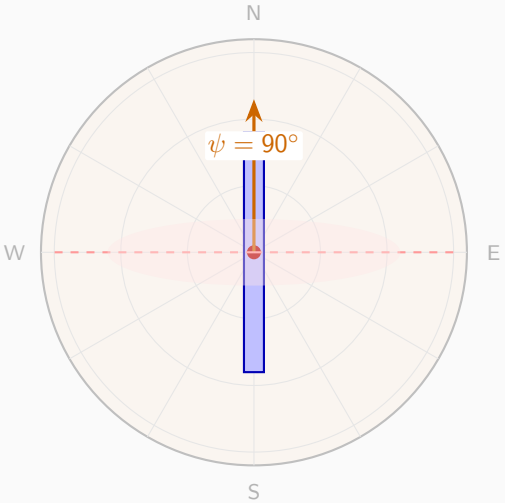
How should we orient our antenna?



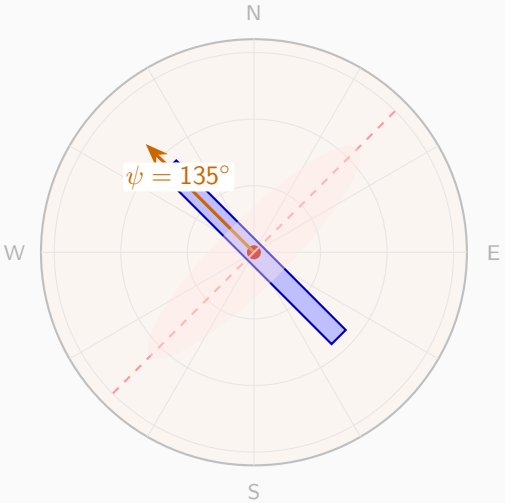
How should we orient our antenna?



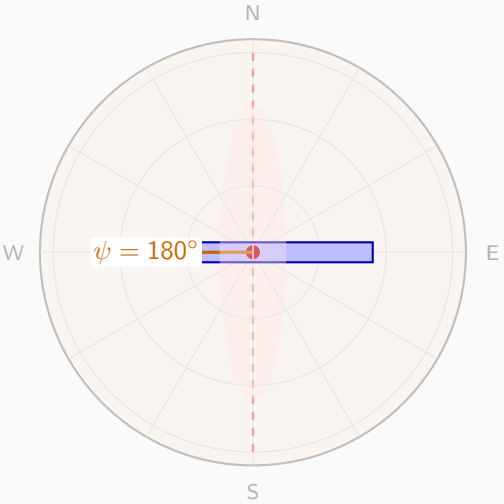
How should we orient our antenna?



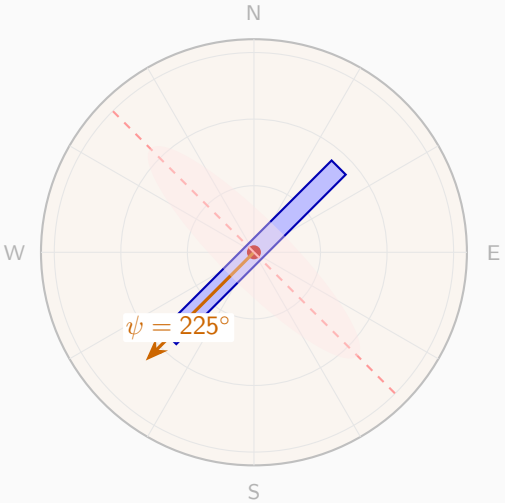
How should we orient our antenna?



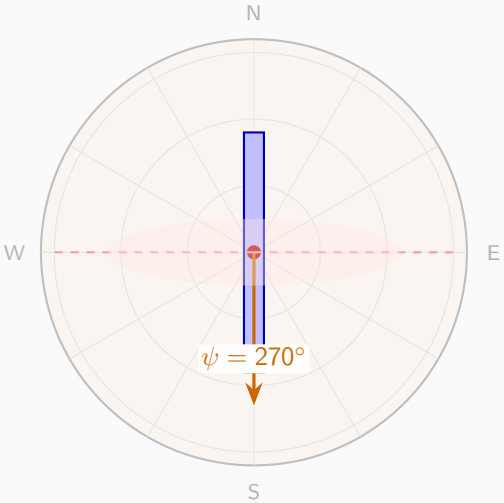
How should we orient our antenna?



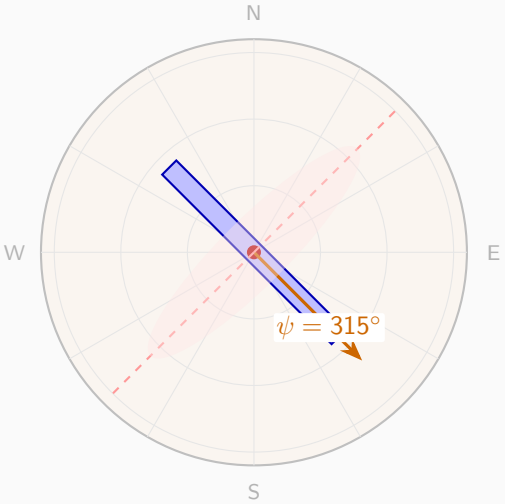
How should we orient our antenna?



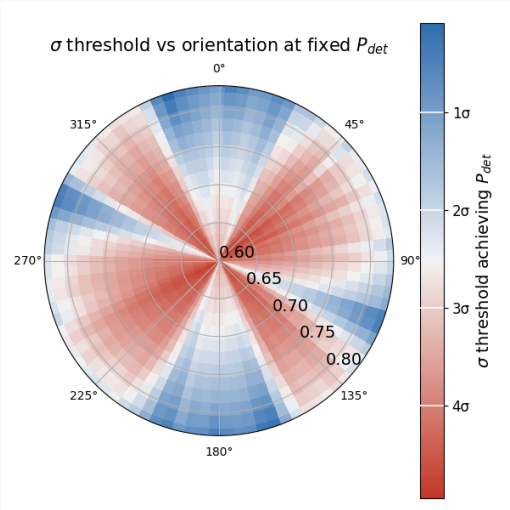
How should we orient our antenna?



How should we orient our antenna?



Can one design choice really move the needle that much?



One night of observation on REACH:

- We vary only antenna orientation.
- Detection probability changes by about **20 percentage points**.
- Four-fold symmetry from dipole beam geometry.

Why this matters

This costs **nothing**. How much would the funders of any state-of-the-art experiment pay for a 20% increase in sensitivity?

Applications

What do you need to use cNBRE?

All you need is a **forward simulator** and a **design parameter** you want to optimise.

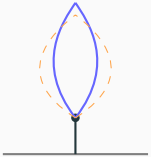
Most experiments that do parameter estimation **already have** a forward model inside their likelihood.

1. Define your two models (M_1 vs M_0).
2. Identify your design parameters α.
3. Simulate training data across the design space.
4. Train a single cNBRE network.

The simulator does not need to be differentiable, fast, or GPU-based.

It just needs to produce realistic mock data for any α.

Instrument design



Antenna geometry

$\alpha =$ beam parameters

Optimise: P_{det}

Calibration hardware

$\alpha = \ell$ (cable length)

Optimise: P_{det}

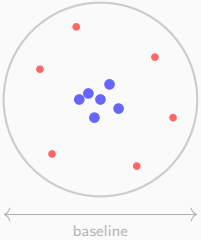
Component tolerance

$\alpha = \delta_{\mathrm{VNA}}, \delta_{\mathrm{LNA}}, \ldots$

Optimise: P_{det}

These parameters are fixed **before construction**. Getting them wrong is expensive to undo. cNBRE lets you explore the full design space from a single training run.

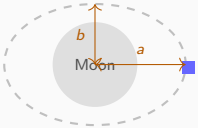
Array and orbit design



Array configuration

$\alpha =$ baseline distribution

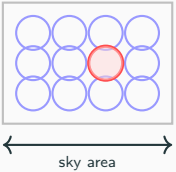
Optimise: P_{det}



Lunar orbit

$\alpha = (a, b, i, \ldots)$

Optimise: P_{det}



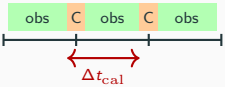
Integration per pointing

$\alpha = t_{\mathrm{point}}$

Optimise: P_{det}

Array layout, orbit parameters, and survey strategy all involve high-dimensional trade-offs where exhaustive grid search is intractable.

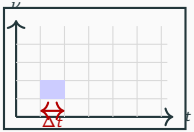
Operations and scheduling



Calibration cadence

$\alpha = \Delta t_{\mathrm{cal}}$

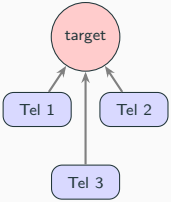
Optimise: P_{det}



Correlator integration

$\alpha = \Delta t_{\mathrm{dump}}$

Optimise: P_{det}



Multi-telescope scheduling

$\alpha =$ time allocation

Optimise: P_{det}

Operational parameters can be re-optimised throughout the lifetime of the experiment – cNBRE amortises the cost of exploring new strategies.

Key takeaways

1. **Optimise one or several design parameters simultaneously.**

2. Combine design optimisation with physical models of the signal you are searching for.

3. Significantly increases the chance of achieving science goals, and is highly cost-effective.

Key takeaways

1. **Optimise one or several design parameters simultaneously.**

2. **Combine design optimisation with physical models of the signal you are searching for.**

3. Significantly increases the chance of achieving science goals, and is highly cost-effective.

Key takeaways

1. **Optimise one or several design parameters simultaneously.**

2. **Combine design optimisation with physical models of the signal you are searching for.**

3. **Significantly increases the chance of achieving science goals, and is highly cost-effective.**

Thanks for listening!

Questions?

Slides and source:

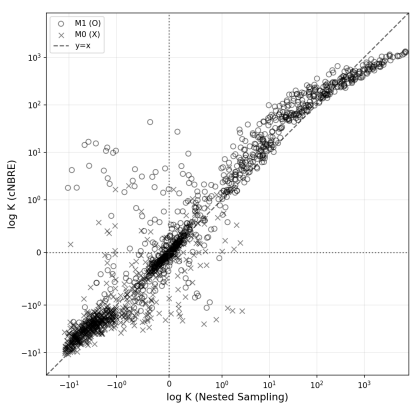
github.com/samleeney/Talks

Email:

sakl2@cam.ac.uk

QR code to the talk repository

Does the amortised answer agree with nested sampling?



- cNBRE broadly tracks nested sampling on held-out simulations.
- For null-model data, cNBRE is more conservative: fewer false positives.
- Training: $\mathcal{O}(100)$ **GPU-hours once**. Evaluation: **milliseconds**.

Nested sampling remains the reference calculation.

cNBRE matters because it amortises the expensive step.

Derivation: optimal network output is the log-Bayes factor

Minimise the expected exponential loss:

$$\mathcal{J}[f] = \int \left[p(\mathbf{d}|M_1)\, e^{-f(\mathbf{d})/2} + p(\mathbf{d}|M_0)\, e^{f(\mathbf{d})/2} \right] \mathrm{d}\mathbf{d}.$$

Set the functional derivative to zero:

$$\frac{\delta \mathcal{J}}{\delta f} = -\tfrac{1}{2}\, p(\mathbf{d}|M_1)\, e^{-f^*/2} + \tfrac{1}{2}\, p(\mathbf{d}|M_0)\, e^{f^*/2} = 0.$$

Rearranging:

$$e^{f^*(\mathbf{d})} = \frac{p(\mathbf{d}|M_1)}{p(\mathbf{d}|M_0)} = K(\mathbf{d}) \qquad \Longrightarrow \qquad \boxed{f^*(\mathbf{d}) = \log K(\mathbf{d})}.$$

Conditioning on a design parameter α: replace $p(\mathbf{d}|M_i) \to p(\mathbf{d}|M_i, \alpha)$ throughout. The same derivation gives

$$\boxed{f^*(\mathbf{d}, \alpha) = \log K(\mathbf{d} \mid \alpha) = \log \frac{p(\mathbf{d}|M_1, \alpha)}{p(\mathbf{d}|M_0, \alpha)}}.$$